

Skill: Apple Style Photo Editor

Para agentes Claude Code — Ingeniería inversa de fotografía Apple

🍏 Principios Fotográficos Apple (Reverse Engineered)

Basado en análisis de campañas oficiales de Apple (MacBook Pro, iPhone, etc.):

1. Iluminación Suave Direccional

- Fuente de luz única dominante (ventana lateral o estudio controlado)
- Sin sombras duras — siempre hay luz de relleno
- Luz natural preferida: color temperature ~5500K

2. Sombras Alzadas (Shadow Lift)

- Las sombras **nunca llegan a negro puro** (valor mínimo ~15-25/255)
- Crea sensación de "aire" y profundidad sin aplastar detalles
- Fórmula: $\text{shadow_out} = \text{shadow_in} + \text{lift} * (1 - \text{shadow_in} / 128)$

3. Altas Luces Protegidas (Highlight Roll-off)

- Los brillos se comprimen suavemente — nunca quemados
- Curva de tono tipo cinematográfico (roll-off en altas luces)
- Fórmula: $\text{highlight_out} = \text{highlight_in} - \text{protect} * ((\text{highlight_in} - 128) / 127)$

4. Split Toning Teal-Naranja (Firma Apple)

- **Sombras:** empuje hacia teal/azul-verde frío
- **Altas luces:** empuje hacia ámbar/naranja cálido
- Crea contraste de color complementario que da "profundidad cinematográfica"
- Intensidad: ~4-6% de mezcla de color

5. Temperatura Cálida en Piel

- White balance ligeramente cálido (+8 a +20K equivalente)
- Tonos de piel dorados/ámbar — nunca fríos o azulados
- Canal rojo +0.55, verde +0.09, azul -0.55 (por unidad de warmth)

6. Vibrance sobre Saturación (Smart Color)

- Vibrance boost a colores desaturados, protege ya-saturados
- Nunca saturación global agresiva — evita skin tone naranja

- Fórmula vibrance: $\text{sat_out} = \text{sat} + \text{vibrance} * (1 - \text{sat_current})$

7. Profundidad de Campo Shallow (DoF)

- Sujeto ultra nítido, fondo cremoso (bokeh suave)
- Simulación: blur radial que aumenta desde el centro
- Blur amount: $\text{strength} * 16 \text{ px}$ en capa exterior

8. Viñeta Sutil

- Oscurecimiento de bordes muy suave (no amateur)
- Gradiente radial: transparente 0-52%, oscuro solo en extremos
- Opacidad máxima: ~0.6-0.9 del valor de vignette

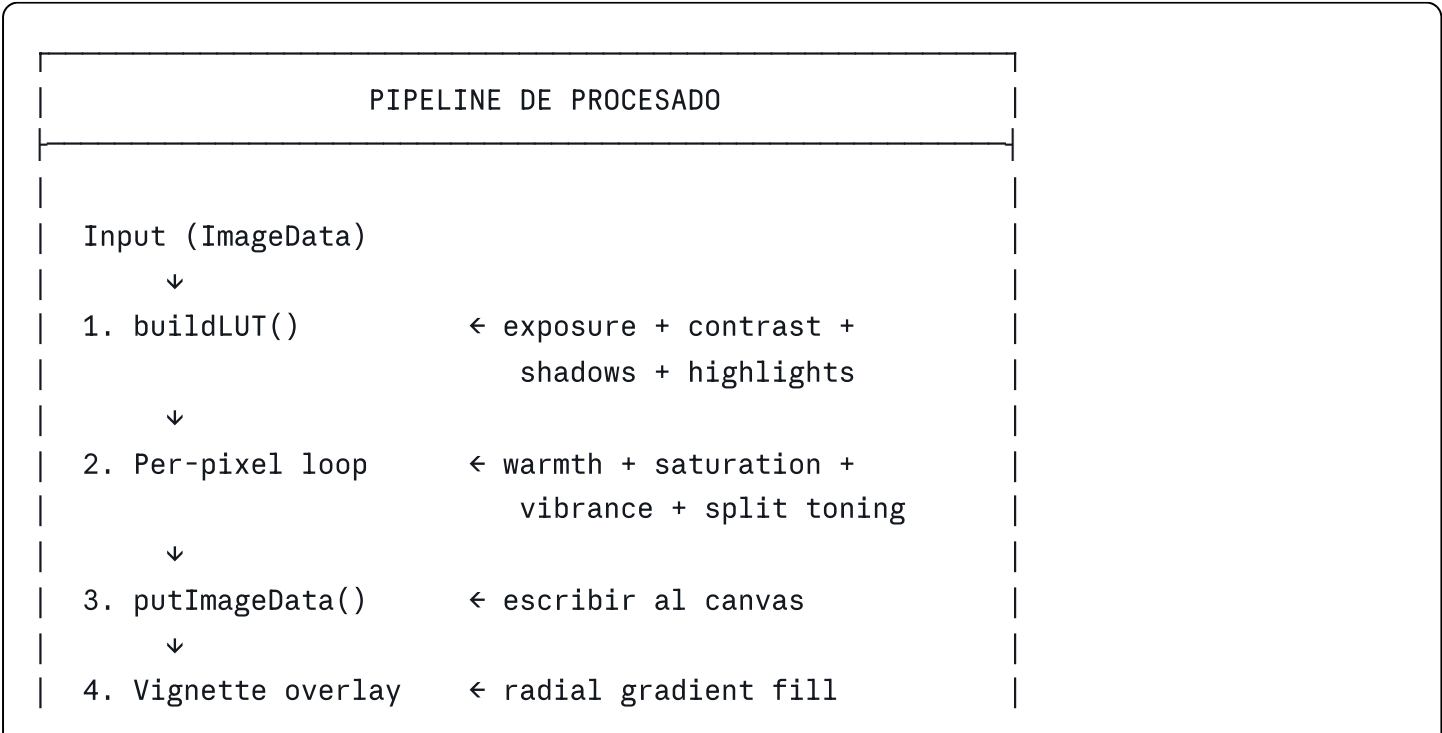
9. Claridad Local (Clarity)

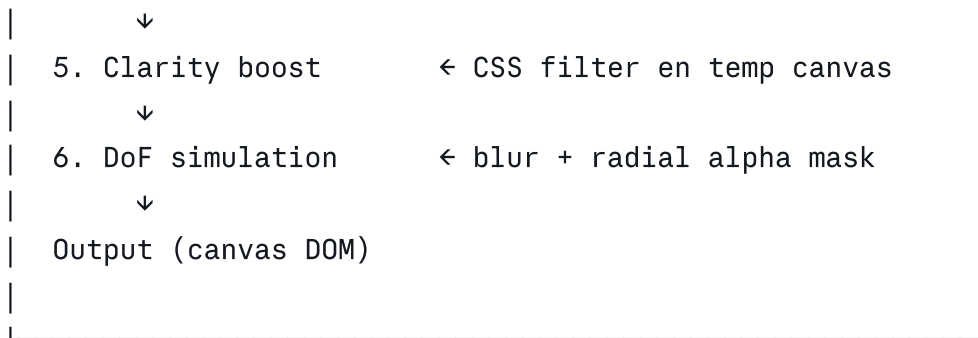
- Contraste de medios tonos / detalles de textura
- Implementado como $\text{contrast}(1 + \text{clarity} * 0.32)$ CSS filter
- Hace que materiales (metal, tela, piel) se vean tangibles

10. Curva S Signature

- S-curve clásica: sombras alzadas + midtones ricos + highlights protegidos
- Contraste: $128 + (v - 128) * (1 + \text{contrast} * 0.9)$
- Exposición multiplicativa: $v * 2^{(\text{exposure} * 1.3)}$

🏠 Arquitectura del Editor





⚙️ Funciones Core

buildLUT({ exposure, contrast, shadows, highlights })

Precalcula una tabla de 256 valores para transformaciones de tono. **Performance:** $O(256)$ — llamar una vez por renderizado.

```
js

function buildLUT({ exposure, contrast, shadows, highlights }) {
  const lut = new Uint8ClampedArray(256);
  for (let i = 0; i < 256; i++) {
    let v = i;
    v = v * Math.pow(2, exposure * 1.3);           // Exposición
    if (v < 128 && shadows > 0)
      v += shadows * 58 * (1 - v / 128);           // Shadow lift
    if (v > 128 && highlights < 0)
      v += highlights * 58 * ((v - 128) / 127);    // Highlight protection
    v = 128 + (v - 128) * (1 + contrast * 0.9);    // Contraste S-curve
    lut[i] = Math.max(0, Math.min(255, Math.round(v)));
  }
  return lut;
}
```

applyDoF(ctx, sharpCanvas, w, h, strength)

Simula profundidad de campo con blur radial creciente.

```
js
```

```
function applyDoF(ctx, sharp, w, h, strength) {
  // 1. Crear versión borrosa
  const blurC = document.createElement('canvas');
  const bCtx = blurC.getContext('2d');
  bCtx.filter = `blur(${Math.round(strength * 18)}px)`;
  bCtx.drawImage(sharp, 0, 0);

  // 2. Máscara: centro transparente → bordes opacos
  const maskC = document.createElement('canvas');
  const mCtx = maskC.getContext('2d');
  mCtx.drawImage(blurC, 0, 0);
  mCtx.globalCompositeOperation = 'destination-in';
  // Gradiente radial: centro=0%, bordes=strength

  // 3. Componer: imagen nítida base + bordes borrosos
  ctx.drawImage(sharp, 0, 0);
  ctx.drawImage(maskC, 0, 0);
}
```

runPipeline(canvas, origCanvas, adj)

Pipeline completo. Lee de origCanvas, escribe a canvas.

🎨 Presets Apple

Preset	Uso	DoF	Warmth	Saturation	Notes
Portrait	Retratos humanos	0.40	+18	+0.05	Máximo bokeh
Lifestyle	Foto editorial/lifestyle	0.18	+10	-0.05	Menos saturación
Product	Hardware/objetos	0	+2	+0.10	Sin DoF, máx claridad
Display	Pantallas (como MacBook ads)	0	0	+0.32	HDR look, vibrante

🔧 Parámetros de Ajuste

Parámetro	Rango	Default	Descripción
exposure	-1 → +1	0	Exposición en stops
contrast	-1 → +1	0	Curva S de contraste

Parámetro	Rango	Default	Descripción
highlights	-1 → 0	0	Protección de altas luces
shadows	0 → +1	0	Alzado de sombras
warmth	-30 → +30	0	Temperatura (K equiv.)
saturation	-0.5 → +0.5	0	Saturación global
vibrance	0 → 0.5	0	Saturación inteligente
clarity	0 → 0.5	0	Contraste local
vignette	0 → 0.8	0	Oscurecimiento de bordes
dof	0 → 1	0	Intensidad de bokeh simulado

Cómo Extender

Agregar un nuevo Preset

```
js
const PRESETS = [
  // ...existing presets
  {
    id: 'mi_preset',
    label: 'Mi Estilo',
    sub: 'Descripción',
    adj: {
      ...DEF, // parte del default
      exposure: 0.08,
      warmth: 25,
      // ...otros ajustes
    }
  }
];
```

Agregar un nuevo Efecto de Pixel

En la función `runPipeline`, dentro del loop `for (let i = 0; i < d.length; i += 4):`

```
js
```

```
// Ejemplo: Efecto "Fade" (matte look)
const fuseAmt = 0.12;
const lum = (r * 0.299 + g * 0.587 + b * 0.114) / 255;
if (lum < 0.10) {
  // Lift mínimo de sombras (efecto fade/matte)
  r = clamp(r + fuseAmt * 255 * (0.10 - lum) / 0.10);
  g = clamp(g + fuseAmt * 255 * (0.10 - lum) / 0.10);
  b = clamp(b + fuseAmt * 255 * (0.10 - lum) / 0.10);
}
```

Agregar un nuevo Slider

```
js

const SLIDERS = [
  // ...existing sliders
  { key: 'fade', label: 'Fade/Matte', min: 0, max: 0.3, step: 0.01 }
];

// Y el default en DEF:
const DEF = { ..., fade: 0 };
```

Stack Técnico

- **React** (hooks: useState, useRef, useEffect, useCallback)
- **Canvas API** con `ImageData` para manipulación per-píxel
- **CSS Filters** para clarity y efectos de blur
- **HTML5 File API** para carga de imágenes
- **requestAnimationFrame** para procesado asíncrono (no bloquea UI)

Optimizaciones Clave

1. **LUT precalculado** — no recalcula curva en cada píxel
 2. **RAF + cancelAnimationFrame** — debounce automático al mover sliders
 3. **Cap de resolución** — máx 860px para tiempo de procesado < 300ms
 4. **Canvases temporales** — clarity y DoF usan canvases separados (no contamina el original)
 5. `willReadFrequently: true` — hint al browser para optimizar `getImageData`
-

Flujo de Estado React

```
img (Image object)
  ↳ useEffect [img, adj, showBefore]
    ↳ showBefore=true → drawImage(orig)
    ↳ showBefore=false → runPipeline(canvas, orig, adj)
      ↳ LUT → perPixel → vignette → clarity → DoF
```

Skill generado por Claude · Basado en reverse engineering de fotografía Apple